

Baza danych kliniki weterynaryjnej

Baza danych została zaprojektowana dla kliniki weterynaryjnej i służy do zarządzania informacjami o właścicielach zwierząt, pacjentach, wizytach, zabiegach oraz receptach. System umożliwia przechowywanie historii leczenia zwierząt, przypisywanie weterynarzy do wizyt oraz dokumentowanie wykonanych zabiegów i wypisanych leków. Projekt został wykonany w Microsoft SQL Server z wykorzystaniem relacyjnego modelu danych, kluczy głównych i obcych, procedur składowanych, funkcji, triggerów oraz widoków. Baza wspiera również analizę danych medycznych i finansowych kliniki oraz eksport danych do formatu JSON.

- Tworzenie bazy danych i przygotowanie środowiska

Poniższy fragment kodu odpowiada za: przełączenie na bazę systemową master, sprawdzenie czy baza danych już istnieje, usunięcie starej wersji bazy, utworzenie nowej bazy danych oraz przełączenie kontekstu pracy na nową bazę.

Kod został przygotowany w sposób umożliwiający wielokrotne uruchamianie całego skryptu bez błędów.

```
USE master;
GO

--sprawdzenie czy baza istnieje
IF EXISTS (SELECT *FROM sys.databases WHERE
name = 'KlinikaWeterynaryjnaDB')
BEGIN

-- zamknięcie połączeń do bazy
ALTER DATABASE KlinikaWeterynaryjnaDB
SET SINGLE_USER
WITH ROLLBACK IMMEDIATE;

-- usunięcie bazy
DROP DATABASE KlinikaWeterynaryjnaDB;

END;
GO

-- tworzenie nowej bazy
CREATE DATABASE KlinikaWeterynaryjnaDB;
GO

-- wejście do bazy
USE KlinikaWeterynaryjnaDB;
GO
```

- Tabela Właściciele

Tabela przechowuje dane właścicieli zwierząt. Zawiera informacje takie jak imię, nazwisko, numer telefonu, adres e-mail oraz adres zamieszkania właściciela.

```
CREATE TABLE Owners (
  owner_id INT IDENTITY(1,1) PRIMARY KEY,
  first_name NVARCHAR(30) NOT NULL,
  last_name NVARCHAR(30) NOT NULL,
  phone NVARCHAR(9) NOT NULL,
  email NVARCHAR(30),
  city NVARCHAR(30) NOT NULL,
  street NVARCHAR(30) NOT NULL,
  postal_code CHAR(6) NOT NULL
```

	owner_id	first_name	last_name	phone	email	city	street	postal_code
1	1	Jan	Kowalski	123456789	jan@gmail.com	Kraków	ul. Długa 10	30-011
2	2	Anna	Nowak	987654321	anna@gmail.com	Katowice	ul. Znajoma 5	40-002
3	3	Piotr	Zieliński	222111333	NULL	Kraków	ul. Szeroka 8	30-111
4	4	Marzena	Mazur	987654311	m.mazur12@wp.pl	Niepołomice	ul. Polna 15	35-102
5	5	Ewelina	Wójcik	123654321	ewelina.w@gmail.com	Kraków	ul. Krawiecka 88	30-123
6	6	Tomasz	Baran	432154321	NULL	Kraków	ul. Brzozowa 4	31-112
7	7	Robert	Dąbrowski	121254321	r.dabrowski@onet.pl	Katowice	ul. Sosnowa 11	40-111
8	8	Katarzyna	Niewiadoma	795132111	katniew@gmail.com	Słomniki	ul. Krakowska 22	36-321
9	9	Julia	Małarczyk	771257327	NULL	Kraków	aleja 29 listopada 32	31-452
10	10	Adrian	Małysz	721357428	NULL	Zielonki	ul. Krakowskie Przedmieście 55	32-078

Rysunek 1. Struktura tabeli Owners

- Tabela Zwierzęta

Tabela Pets przechowuje informacje o zwierzętach należących do właścicieli. Zawiera dane takie jak imię zwierzęcia, gatunek, rasa, data urodzenia, waga oraz płeć. Kolumna gender może przyjmować tylko wartości: "samiec", "samica" lub "nieznana", a domyślnie ustawiana jest wartość "nieznana" w przypadku, gdy podczas dodawania rekordu użytkownik nie poda płci zwierzęcia.

```
CREATE TABLE Pets (
  animal_id INT IDENTITY(1,1) PRIMARY KEY,
  name NVARCHAR(30) NOT NULL,
  species NVARCHAR(20) NOT NULL,
  breed NVARCHAR(30) NULL,
  birth_date DATE NULL,
  weight DECIMAL(5,2),
  gender NVARCHAR(10) NOT NULL DEFAULT 'nieznana'
  CHECK (gender IN
    ('samiec', 'samica', 'nieznana')),
  owner_id INT,
  FOREIGN KEY (owner_id) REFERENCES Owners(owner_id)
);
```

	animal_id	name	species	breed	birth_date	weight	gender	owner_id
1	1	Rex	pies	labrador	2020-05-10	30.50	samiec	1
2	2	Mruczek	kot	dachowiec	NULL	4.20	samiec	2
3	3	Bella	pies	bulldog	2023-04-01	12.00	samica	1
4	4	Kropka	kot	perski	2019-11-05	3.80	samica	3
5	5	Kruszynka	chomik	NULL	NULL	NULL	samica	4
6	6	Cynamon	kot	dachowiec	NULL	4.00	samiec	5
7	7	Puszek	kot	ragdoll	2024-05-12	4.70	samiec	6
8	8	Mamba	kot	bombajski	2023-04-20	4.00	samica	6
9	9	Maks	pies	NULL	NULL	11.00	samiec	7
10	10	Kulka	chomik	NULL	NULL	0.38	samica	8
11	11	Pasztecik	chomik	NULL	NULL	0.45	samiec	8
12	12	Azor	pies	kundel	NULL	15.32	samiec	9
13	13	Nela	pies	maltańczyk	2025-04-10	3.30	nieznana	10

Rysunek 2. Struktura tabeli Pets

- *CHECK (gender IN ('samiec','samica','nieznana'))* - Ograniczenie sprawdzające poprawność danych. Dozwolone są wyłącznie trzy wartości. Próba dodania innej wartości spowoduje wygenerowanie błędu przez SQL Server.

```
-- tu przykład gdy płeć nieznana
INSERT INTO Pets (name, species, breed, birth_date, weight,owner_id) VALUES
('Pasztecik', 'chomik',NULL,NULL, 0.45,8);
```

9	9	Maks	pies	NULL	NULL	11.00	samiec	7
10	10	Kulka	chomik	NULL	NULL	0.38	samica	8
11	11	Pasztecik	chomik	NULL	NULL	0.45	nieznana	8

- Tabela Weterynarze

Tabela przechowuje informacje o weterynarzach pracujących w klinice. Zawiera dane takie jak imię, nazwisko, specjalizacja, numer telefonu oraz data zatrudnienia.

```
CREATE TABLE Vets (
    vet_id INT IDENTITY(1,1) PRIMARY KEY,
    first_name NVARCHAR(30) NOT NULL,
    last_name NVARCHAR(30) NOT NULL,
    specialization NVARCHAR(50) NOT NULL,
    phone NVARCHAR(9) NOT NULL,
    hire_date DATE NOT NULL
);
```

	vet_id	first_name	last_name	specialization	phone	hire_date
1	1	Marek	Nowak	chirurgia	791227333	2015-03-01
2	2	Ewa	Lis	dermatologia	324855166	2020-06-15
3	3	Tomasz	Kaczmarek	ortopedia	127883919	2018-09-10
4	4	Anna	Wójcik	radiologia	772333444	2019-01-20
5	5	Paweł	Mazur	asystent weterynarii	755688717	2019-11-05
6	6	Janusz	Szymański	kardiologia	744628788	2018-10-15

Rysunek 3. Struktura tabeli Vets

- Tabela Wizyty

Tabela Visits przechowuje informacje o wizytach pacjentów, takie jak termin wizyty, dane pacjenta oraz lekarza. Każdy rekord reprezentuje jedną wizytę. Kolumna status określa aktualny stan wizyty i może przyjmować tylko wartości: "zaplanowana", "zakończona" lub

"anulowana". Domyślnie każda nowa wizyta otrzymuje status "zaplanowana" w chwili rezerwacji wizyty. Dodatkowe ograniczenie sprawdza, czy zakończona wizyta ma przypisanego weterynarza, czyli wiadomo, który lekarz przeprowadził wizytę. Wizyty zaplanowane lub anulowane mogą nie mieć jeszcze przypisanego lekarza, ponieważ przy planowaniu specjalista może jeszcze nie być przydzielony.

```
CREATE TABLE Visits (
    visit_id INT IDENTITY(1,1) PRIMARY KEY,
    animal_id INT NOT NULL,
    vet_id INT NULL,
    visit_date DATETIME NOT NULL,
    reason NVARCHAR(70) NOT NULL,
    notes NVARCHAR(255) NULL,
    status NVARCHAR(20) DEFAULT 'zaplanowana'
    CHECK (status IN
('zaplanowana','zakończona','anulowana')),

    FOREIGN KEY (animal_id) REFERENCES Pets(animal_id),
    FOREIGN KEY (vet_id) REFERENCES Vets(vet_id),

    CHECK (
(status = 'zakończona' AND vet_id IS NOT NULL)
OR
(status IN ('zaplanowana','anulowana'))
)
);
```

	visit_id	animal_id	vet_id	visit_date	reason	notes	status
1	1	1	5	2025-04-10 10:00:00.000	szczepienie	brak uwag	zakończona
2	2	2	2	2025-04-11 12:00:00.000	alergia	wysypka na skórze	zakończona
3	3	3	1	2025-04-15 09:00:00.000	kontrola	NULL	zakończona
4	4	4	3	2025-04-16 11:00:00.000	uraz łapy	lekkie kulawienie	zakończona
5	5	5	1	2025-05-16 13:00:00.000	operacja	usunięcie guza	zakończona
6	6	6	NULL	2025-05-18 14:00:00.000	badanie ogólne	NULL	anulowana
7	7	7	4	2025-05-16 11:00:00.000	diagnostyka	badanie krwi oraz badanie ogólne	zakończona
8	8	8	4	2025-05-18 14:00:00.000	badanie ogólne	NULL	anulowana
9	9	9	3	2025-05-16 11:00:00.000	uraz łapy	uraz spowodowany niefortunnym skokiem	zakończona
10	10	10	NULL	2025-05-18 14:00:00.000	badanie ogólne	NULL	zaplanowana
11	11	11	5	2025-05-20 13:30:00.000	badanie ogólne	brak uwag	zaplanowana
12	12	12	5	2025-05-21 10:00:00.000	szczepienie	NULL	zaplanowana
13	13	13	NULL	2025-05-21 11:30:00.000	kontrola - opatrzenie rany	rana w fazie gojenia	zaplanowana
14	14	4	3	2025-05-22 10:00:00.000	kontrola - uraz łapy	lekkie kulawienie	zaplanowana
15	15	9	NULL	2025-05-22 11:00:00.000	kontrola - uraz łapy	uraz spowodowany niefortunnym skokiem	zaplanowana
16	16	5	1	2025-05-22 12:00:00.000	kontrola po operacji chomika	NULL	zaplanowana

Rysunek 4. Struktura tabeli Visits

- Tabela Zabiegi

Tabela Treatments przechowuje informacje o wykonanych zabiegach podczas wizyty. Zawiera nazwę zabiegu, jego opis, notatki lekarza, czas trwania oraz koszt. Każdy zabieg jest powiązany z konkretną wizytą, który faktycznie został wykonany podczas wizyty.

```

CREATE TABLE Treatments (
    treatment_id INT IDENTITY(1,1) PRIMARY KEY,
    visit_id INT NOT NULL,
    name NVARCHAR(50) NOT NULL,
    description NVARCHAR(255) NULL,
    notes NVARCHAR(255) NULL,
    duration INT NOT NULL,
    cost DECIMAL(7,2) NOT NULL

    FOREIGN KEY (visit_id) REFERENCES
    Visits(visit_id)
);

```

	treatment_id	visit_id	name	description	notes	duration	cost
1	1	1	szczepienie	podanie szczepionki przeciw wściekliźnie	Zwierzę było spokojne. Podano szczepionkę. Brak rea...	40	50.00
2	2	2	alergia	podanie leków na alergię	Wysypka zniknęła częściowo. Zalecana kontrola po tyg...	30	40.00
3	3	3	kontrola	kontrola stanu zdrowia	Zwierzę w dobrej kondycji. Brak uwag.	60	30.00
4	4	4	uraz łapy	wykonanie zdjęcia RTG łapy	Rana oczyszczona i zabandażowana. Pacjent był nies...	90	90.00
5	5	5	operacja	Przeprowadzono operację guza u chomika. Pacjent ...	NULL	120	150.00
6	6	7	diagnostyka	Standardowe badanie ogólne.	Informacja o profilaktyce.	60	15.00
7	7	9	uraz łapy	Oczyszczenie i zdezynfekowanie rany. Założenie opa...	Uraz spowodowany niefortunnym skokiem. Rana oczy...	100	110.00

Rysunek 5. Struktura tabeli Treatments

- Tabela Recepty

Tabela Prescriptions przechowuje informacje o przepisanych lekach po wykonanym zabiegu. Zawiera nazwę leku, dawkowanie, czas trwania leczenia oraz dodatkowe uwagi lekarza. Każda recepta jest powiązana z konkretnym zabiegiem. Jeden zabieg może posiadać wiele recept np. maść, lek przeciwbólowy. Dlatego w tabeli może występować wiele rekordów przypisanych do tego samego zabiegu.

```

CREATE TABLE Prescriptions (
    prescription_id INT IDENTITY(1,1) PRIMARY KEY,
    treatment_id INT NOT NULL,
    medicine_name NVARCHAR(100) NOT NULL,
    dosage NVARCHAR(50) NOT NULL,
    duration_days INT NOT NULL,
    notes NVARCHAR(MAX) NULL,

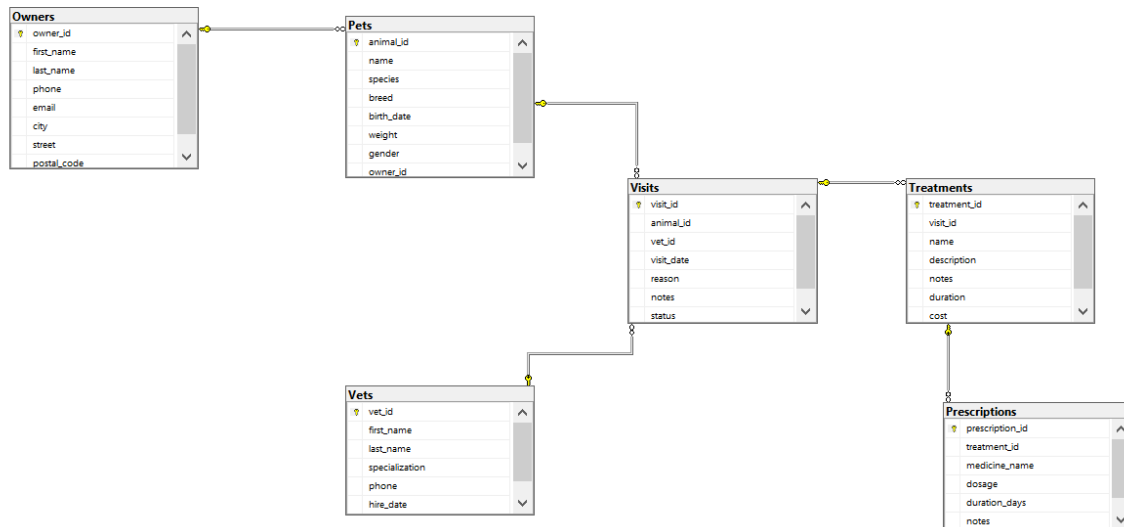
    FOREIGN KEY (treatment_id) REFERENCES
    Treatments(treatment_id)
);

```

	prescription_id	treatment_id	medicine_name	dosage	duration_days	notes
1	1	2	Dolvit Allergy	2x dziennie po 1 tabletkę	7	Podawać po posiłku.
2	2	4	RemediCann	2x dziennie	10	Maść przeciwzapalna. Nanieść cienką warstwę na ra...
3	3	4	Meloksykam	1x dziennie	5	Środek przeciwbólowy. Podawać w razie bólu.
4	4	5	Antybiotyk VetCure	2x dziennie po 0.5 ml	7	Podawać po jedzeniu. Obserwować stan zwierzęcia.
5	5	5	Meloksykam	1x dziennie	5	Podawać zgodnie z zaleceniami lekarza.
6	6	6	Dolfos Vetcal D3	1x dziennie	14	Witamina D. Podawać razem z jedzeniem.
7	7	7	Beaphar Wound Maść	2x dziennie	10	Maść łagodzącą stosować na oczyszczoną ranę. Ko...

Rysunek 6. Struktura tabeli Prescriptions

Diagram



Rysunek 7. Diagram

Procedury

- Procedura FinishVisit

Procedura służy do zakończenia wizyty weterynaryjnej oraz jednoczesnego dodania wykonanego zabiegu do tabeli Treatments. Realizuje ona logikę biznesową kliniki weterynaryjnej, czyli tylko wizyty o statusie zaplanowana mogą zostać zakończone. Po zakończeniu wizyty musi zostać przypisany lekarz, następnie dodawany jest opis wykonanego zabiegu. W procedurze jest użyta transakcja. Oznacza to, że wszystkie operacje wykonają się razem, albo zapiszą się wszystkie zmiany albo w przypadku błędu żadna zmiana nie zostanie zapisana. Przykład: jeśli status wizyty został zmieniony, ale dodanie zabiegu się nie udało, to zmiana statusu również zostanie cofnięta.

```

CREATE PROCEDURE FinishVisit
(
    @visit_id INT,
    @vet_id INT,
    @treatment_name NVARCHAR(50),
    @description NVARCHAR(255),
    @treatment_notes NVARCHAR(255),
    @duration INT,
    @cost DECIMAL(7,2)
)
AS
BEGIN
    BEGIN TRANSACTION;
    BEGIN TRY

        IF NOT EXISTS (SELECT 1 FROM Visits WHERE visit_id = @visit_id AND status = 'zaplanowana')
        BEGIN
            THROW 50001, 'Wizyta nie jest zaplanowana, może być anulowana lub zakończona. Brak możliwości dodania leczenia.', 1;
        END;

        UPDATE Visits
        SET
            vet_id = @vet_id,
            status = 'zakończona'
        WHERE visit_id = @visit_id;

        INSERT INTO Treatments(visit_id, name, description, notes, duration, cost)
        VALUES (@visit_id, @treatment_name, @description, @treatment_notes,
        @duration, @cost);

        COMMIT TRANSACTION;

    END TRY
    BEGIN CATCH

        ROLLBACK TRANSACTION;

        THROW;

    END CATCH
END;
GO

```

- Sprawdzenie poprawne

```

EXECUTE FinishVisit
    @visit_id = 10,
    @vet_id = 4,
    @treatment_name = 'badanie ogólne',
    @description = 'kontrola stanu zdrowia',
    @treatment_notes = NULL,
    @duration = 60,
    @cost = 30;

SELECT * FROM Visits;
SELECT * FROM Treatments;

```

- Visits
(Przed)

id	vet_id	status	description	timestamp	name	notes	status
10	10	10	NULL	2025-05-18 14:00:00.000	badanie ogólne	NULL	zaplanowana

Rysunek 8. Przykład rekordu tabeli Visits przed modyfikacją

(Po)

	visit_id	animal_id	vet_id	visit_date	reason	notes	status
1	10	10	4	2025-05-18 14:00:00.000	badanie ogólne	NULL	zakończona

Rysunek 9. Przykład rekordu tabeli Visits po modyfikacji

- Treatments

	treatment_id	visit_id	name	description	notes	duration	cost
1	8	10	badanie ogólne	kontrola stanu zdrowia	NULL	60	30.00

Rysunek 10. Dodanie rekordu do tabeli Treatments

- Sprawdzenie błędne

```
EXECUTE FinishVisit
    @visit_id = 8, -- wizyta anulowana
    @vet_id = 4,
    @treatment_name = 'badanie ogólne',
    @description = 'kontrola stanu zdrowia',
    @treatment_notes = NULL,
    @duration = 60,
    @cost = 30;
```

Komunikat:

Msg 50001, Level 16, State 1, Procedure FinishVisit, Line 22 [Batch Start Line 269]
Wizyta nie jest zaplanowana, może być anulowana lub zakończona. Brak możliwości dodania leczenia.

Rysunek 11. Komunikat błędu

- Procedura podrzędna AddPrescription i główna FinishVisitWithPrescription

Procedura AddPrescription odpowiada za dodanie recepty do tabeli Prescriptions.

```
CREATE PROCEDURE AddPrescription
(
    @treatment_id INT,
    @medicine_name NVARCHAR(100),
    @dosage NVARCHAR(50),
    @duration_days INT,
    @notes NVARCHAR(MAX)
)
AS
BEGIN
    BEGIN TRY

        INSERT INTO Prescriptions (treatment_id, medicine_name,
dosage, duration_days, notes) VALUES
            (@treatment_id, @medicine_name, @dosage,
@duration_days, @notes);

    END TRY
    BEGIN CATCH
        THROW; -- przekazuje błąd do procedury nadrzędnej
    END CATCH
END;
GO
```

Procedura FinishVisitWithPrescription służy do zakończenia wizyty, przypisania lekarza, dodania zabiegu lub opcjonalnego dodania recepty.

Procedura pokazuje przykład transakcji, obsługi błędów i wywołania jednej procedury z poziomu drugiej procedury.

```
CREATE PROCEDURE FinishVisitWithPrescription
(
    @visit_id INT,
    @vet_id INT,
    @treatment_name NVARCHAR(50),
    @description NVARCHAR(255),
    @treatment_notes NVARCHAR(255),
    @duration INT,
    @cost DECIMAL(7,2),
    @medicine_name NVARCHAR(100) = NULL,
    @dosage NVARCHAR(50) = NULL,
    @duration_days INT = NULL,
    @prescription_notes NVARCHAR(MAX) = NULL
)
AS
BEGIN
    BEGIN TRANSACTION;

    BEGIN TRY
        DECLARE @treatment_id INT;

        IF NOT EXISTS (SELECT 1 FROM Visits WHERE visit_id = @visit_id AND status =
'zaplanowana')
        BEGIN
            THROW 50001, 'Wizyta nie jest zaplanowana.', 1;
        END;

        UPDATE Visits
        SET vet_id = @vet_id,
            status = 'zakończona'
        WHERE visit_id = @visit_id;

        INSERT INTO Treatments (visit_id, name, description, notes, duration, cost) VALUES
        (@visit_id, @treatment_name, @description, @treatment_notes, @duration, @cost);
```

```
SET @treatment_id = SCOPE_IDENTITY();

    -- Wwołanie innej procedury
    IF @medicine_name IS NOT NULL
    BEGIN
        EXEC AddPrescription
            @treatment_id = @treatment_id,
            @medicine_name = @medicine_name,
            @dosage = @dosage,
            @duration_days = @duration_days,
            @notes = @prescription_notes;
    END;

    COMMIT TRANSACTION;

END TRY
BEGIN CATCH

    ROLLBACK TRANSACTION;

    THROW 50000, 'Operacja nie powiodła się, sprawdź dane wizyty lub recepty.', 1;

END CATCH
END;
GO
```

- Sprawdzenie bez recepty

```
EXEC FinishVisitWithPrescription
  @visit_id = 12,
  @vet_id = 5,
  @treatment_name = 'szczepienie',
  @description = 'podanie szczepionki przeciw wściekliznie',
  @treatment_notes = 'Zwierzę było niespokojne.',
  @duration = 40,
  @cost = 50.00;
```

- Visits

(Przed)

12	12	12	5	2025-05-21 10:00:00.000	szczepienie	NULL	zaplanowana
----	----	----	---	-------------------------	-------------	------	-------------

Rysunek 12. Przykład rekordu tabeli Visits przed modyfikacją

(Po)

12	12	12	5	2025-05-21 10:00:00.000	szczepienie	NULL	zakończona
----	----	----	---	-------------------------	-------------	------	------------

Rysunek 13. Przykład rekordu tabeli Visits po modyfikacji

- Treatments

8	8	12	szczepienie	podanie szczepionki przeciw wściekliznie	Zwierzę było niespokojne.	40	50.00
---	---	----	-------------	--	---------------------------	----	-------

Rysunek 14. Dodanie rekordu do tabeli Treatments

- Sprawdzenie z receptą

```
EXEC FinishVisitWithPrescription
  @visit_id = 13,
  @vet_id = 3,
  @treatment_name = 'kontrola',
  @description = 'kontrola w celu opatrzenie rany',
  @treatment_notes = 'Rana pacjenta jest w dobrej kondycji.
Brak nieprawidłowości.Przypisanie maści.',
  @duration = 30,
  @cost = 70.00,
  @medicine_name = 'RemediCann',
  @dosage = '2x dziennie',
  @duration_days = 7,
  @prescription_notes = 'Maść stosować na oczyszczoną ranę.';
```

- Visits

(Przed)

13	13	13	NULL	2025-05-21 11:30:00.000	kontrola - opatrzenie rany	rana w fazie gojenia	zaplanowana
----	----	----	------	-------------------------	----------------------------	----------------------	-------------

Rysunek 15. Przykład rekordu tabeli Visits przed modyfikacją

(Po)

13	13	13	3	2025-05-21 11:30:00.000	kontrola - ...	rana w fazie goje...	zakończona
----	----	----	---	-------------------------	----------------	----------------------	------------

Rysunek 16. Przykład rekordu tabeli Visits po modyfikacji

- Treatments

9	9	13	kontrola	kontrola w celu opatrzenie rany	Rana pacjenta jest w dobrej kondycji. Brak nieprawidlo...	30	70.00
---	---	----	----------	---------------------------------	---	----	-------

Rysunek 17. Dodanie rekordu do tabeli Treatments

- Prescriptions

	prescription_id	treatment_id	medicine_name	dosage	duration_days	notes
1	8	9	RemediCann	2x dziennie	7	Maść stosować na oczyszczoną ranę.

Rysunek 18. Dodanie rekordu do tabeli Prescriptions

- Procedura VetStatistics

Procedura VetStatistics służy do generowania statystyk dotyczących pracy konkretnego weterynarza. Na podstawie zakończonych wizyt procedura oblicza liczbę wykonanych wizyt, łączny dochód wygenerowany przez lekarza, średni koszt wizyty.

```
CREATE PROCEDURE VetStatistics
(
    @vet_id INT
)
AS
BEGIN
    -- sprawdzenie czy weterynarz ma zakończone wizyty
    IF NOT EXISTS (SELECT 1 FROM Visits WHERE vet_id = @vet_id AND status = 'zakończona')
    BEGIN
        THROW 50010, 'Brak danych: weterynarz nie istnieje lub nie ma zakończonych wizyt.', 1;
    END;

    SELECT
        v.vet_id,
        COUNT(v.visit_id) AS total_visits,
        SUM(t.cost) AS total_income,
        AVG(t.cost) AS avg_visit_cost
    FROM Visits v
    JOIN Treatments t ON v.visit_id = t.visit_id
    WHERE v.vet_id = @vet_id AND v.status = 'zakończona'
    GROUP BY v.vet_id;

END;
GO
```

- Sprawdzenie

```
EXEC VetStatistics
    @vet_id =2;
```

	vet_id	total_visits	total_income	avg_visit_cost
1	2	1	40.00	40.000000

Rysunek 19. Wynik procedury VetStatistics

- Funkcja skalarna

Obliczenie wieku zwierzęcia. Na podstawie daty urodzenia i aktualnej daty.

```

CREATE FUNCTION AnimalAge
(
    @birth_date DATE
)
RETURNS INT
AS
BEGIN

    DECLARE @age INT;
    SET @age = DATEDIFF(YEAR, @birth_date, GETDATE());
    RETURN @age;

END;
GO

```

```

SELECT name, dbo.AnimalAge(birth_date) AS age
FROM Pets;

```

	name	age
1	Rex	6
2	Mruczek	NULL
3	Bella	3
4	Kropka	7
5	Kruszynka	NULL
6	Cynamon	NULL
7	Puszek	2
8	Mamba	3
9	Maks	NULL
10	Kulka	NULL
11	Pasztecik	NULL
12	Azor	NULL
13	Nela	1

Rysunek 20. Wynik funkcji AnimalAge

- Funkcja tabelaryczna

Historia wizyt konkretnego zwierzęcia.

```
CREATE FUNCTION GetAnimalVisits
(
    @animal_id INT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        visit_id,
        visit_date,
        reason,
        status
    FROM Visits
    WHERE animal_id = @animal_id

    UNION ALL

    SELECT
        NULL,
        NULL,
        'BRAK WIZYT',
        NULL
    WHERE NOT EXISTS (
        SELECT 1 FROM Visits WHERE animal_id = @animal_id
    )
);
GO
```

```
SELECT * FROM dbo.GetAnimalVisits(4);
```

	visit_id	visit_date	reason	status
1	4	2025-04-16 11:00:00.000	uraz łapy	zakończona
2	14	2025-05-22 10:00:00.000	kontrola - uraz łapy	zaplanowana

Rysunek 21. Wynik funkcji GetAnimalVisits

```
SELECT * FROM dbo.GetAnimalVisits(99);
```

	visit_id	visit_date	reason	status
1	NULL	NULL	BRAK WIZYT	NULL

Rysunek 22. Wynik funkcji GetAnimalVisits z brakiem informacji

- Widok: pełna historia wizyty z zabiegiem

```
CREATE VIEW Visit_With_Treatment AS
SELECT
    v.visit_id,
    v.visit_date,
    v.reason,
    t.name AS treatment_name,
    t.cost,
    t.duration
FROM Visits v
JOIN Treatments t ON v.visit_id = t.visit_id
WHERE v.status = 'zakończona';
GO
```

	visit_id	visit_date	reason	treatment_name	cost	duration
1	1	2025-04-10 10:00:00.000	szczepienie	szczepienie	50.00	40
2	2	2025-04-11 12:00:00.000	alergia	alergia	40.00	30
3	3	2025-04-15 09:00:00.000	kontrola	kontrola	30.00	60
4	4	2025-04-16 11:00:00.000	uraz łapy	uraz łapy	90.00	90
5	5	2025-05-16 13:00:00.000	operacja	operacja	150.00	120
6	7	2025-05-16 11:00:00.000	diagnostyka	diagnostyka	15.00	60
7	9	2025-05-16 11:00:00.000	uraz łapy	uraz łapy	110.00	100
8	12	2025-05-21 10:00:00.000	szczepienie	szczepienie	50.00	40
9	13	2025-05-21 11:30:00.000	kontrola - opatrzenie rany	kontrola	70.00	30

Rysunek 23. Wynik widoku Visit_With_Treatment

- Widok: podsumowanie kosztów leczenia. Pokazuje ile kosztowało leczenie każdego zwierzęcia, sumę kosztów zabiegów.

```
CREATE VIEW AnimalTreatmentCosts AS
SELECT
    p.animal_id,
    p.name AS animal_name,
    SUM(t.cost) AS total_cost,
    COUNT(t.treatment_id) AS treatments_count
FROM Pets p
JOIN Visits v ON p.animal_id = v.animal_id
JOIN Treatments t ON v.visit_id = t.visit_id
GROUP BY p.animal_id, p.name;
GO
```

	animal_id	animal_name	total_cost	treatments_count
1	5	Kruszynka	150.00	1
2	9	Maks	110.00	1
3	4	Kropka	90.00	1
4	13	Nela	70.00	1
5	12	Azor	50.00	1
6	1	Rex	50.00	1
7	2	Mruczek	40.00	1
8	3	Bella	30.00	1
9	7	Puszek	15.00	1

Rysunek 24. Wynik widoku AnimalTreatmentCosts

- Trigger trg_Visit_Delete

Trigger trg_Visit_Delete został utworzony dla tabeli Visits i odpowiada za kontrolowanie procesu usuwania wizyt.

Jego zadaniem jest:

- blokowanie usuwania zakończonych lub anulowanych wizyt,
- blokowanie usuwania wizyt posiadających zabiegi,
- zapisywanie usuniętych wizyt do tabeli logów,
- zapewnienie spójności danych poprzez wykorzystanie transakcji.

```

CREATE TRIGGER trg_Visit_Delete
ON Visits
INSTEAD OF DELETE
AS
BEGIN
    BEGIN TRANSACTION;

    BEGIN TRY

        IF NOT EXISTS (SELECT 1 FROM DELETED)
        BEGIN
            THROW 50003, 'Wizyta nie istnieje.', 1;
        END;

        -- tylko zaplanowane wizyty
        IF EXISTS (SELECT 1 FROM DELETED WHERE status <> 'zaplanowana')
        BEGIN
            THROW 50001, 'Można usuwać tylko wizyty zaplanowane.', 1;
        END;

        -- brak powiązanych zabiegów
        IF EXISTS (SELECT 1 FROM Treatments t JOIN DELETED d
                    ON t.visit_id = d.visit_id)
        BEGIN
            THROW 50002, 'Nie można usunąć wizyty posiadającej zabiegi.', 1;
        END;

        -- log
        INSERT INTO Visit_Log
            (visit_id, animal_id, vet_id, visit_date, reason)
        SELECT
            d.visit_id,
            d.animal_id,
            d.vet_id,
            d.visit_date,
            d.reason
        FROM DELETED d;

        -- delete
        DELETE FROM Visits
        WHERE visit_id IN (SELECT visit_id FROM DELETED);

        COMMIT TRANSACTION;

    END TRY

    BEGIN CATCH

        ROLLBACK TRANSACTION;

        THROW;

    END CATCH

    PRINT 'Usunięto wizytę.';

END;
GO

```

Tabela Visit_Log służy do przechowywania informacji o usuniętych wizytach. Dzięki temu możliwe jest śledzenie historii usunięć, dane nie są całkowicie tracone, można sprawdzić kto i kiedy usunął wizytę.

```

CREATE TABLE Visit_Log (
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    visit_id INT,
    animal_id INT,
    vet_id INT,
    visit_date DATETIME,
    reason NVARCHAR(70),
    deleted_at DATETIME DEFAULT GETDATE()
);
GO

```

- Sprawdzenia
Wizyta anulowana

```
DELETE FROM Visits WHERE visit_id = 1;
```

Msg 50001, Level 16, State 1, Procedure trg_Visit_Delete, Line 26 [Batch Start Line 632]
Można usuwać tylko wizyty zaplanowane.

Rysunek 24. Komunikat błędu

Poprawne

```
DELETE FROM Visits WHERE visit_id = 14;
```

	log_id	visit_id	animal_id	vet_id	visit_date	reason	deleted_at
1	1	14	4	3	2025-05-22 10:00:00.000	kontrola - uraz łapy	2026-06-08 09:21:06.860

Rysunek 25. Przykład działania triggera

- Trigger trg_Treatment_Insert

Trigger został utworzony dla tabeli Treatments i odpowiada za automatyczne logowanie nowych zabiegów dodawanych do systemu. Jego zadaniem jest monitorowanie operacji INSERT, zapisywanie informacji o nowo dodanych zabiegach i tworzenie historii operacji w tabeli logów.

```
CREATE TRIGGER trg_Treatment_Insert
ON Treatments
AFTER INSERT
AS
BEGIN
    INSERT INTO Treatment_Log (treatment_id, visit_id, name, cost)
    SELECT
        i.treatment_id,
        i.visit_id,
        i.name,
        i.cost
    FROM INSERTED i;
END;
```

Tabela Treatment_Log służy do przechowywania historii dodanych zabiegów. Dzięki temu możliwe jest śledzenie operacji wykonywanych w systemie, kontrolowanie dodawanych zabiegów, tworzenie prostego mechanizmu audytu danych.

```
CREATE TABLE Treatment_Log (
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    treatment_id INT,
    visit_id INT,
    name NVARCHAR(50),
    cost DECIMAL(7,2),
    created_at DATETIME DEFAULT GETDATE()
);
GO
```

- Sprawdzenia

```
INSERT INTO Treatments (visit_id, name, description, notes, duration, cost)
VALUES (10, 'badanie ogólne', 'standardowe badanie', 'NULL', 60, 30);
```

	log_id	treatment_id	visit_id	name	cost	created_at
1	1	8	10	badanie ogólne	30.00	2026-06-08 09:35:19.743

Rysunek 26. Przykład działania triggera

	treatment_id	visit_id	name	description	notes	duration	cost
1	8	10	badanie ogólne	standardowe badanie	NULL	60	30.00

Rysunek 27. Przykładowy dodany rekord do tabeli Treatments

- Zapytanie z wykorzystaniem JOIN

Zapytanie umożliwia wyświetlenie: właściciela zwierzęcia, danych zwierzęcia, historii wizyt, wykonanych zabiegów, przepisanych leków. Jest to przykład połączenia danych relacyjnych z wielu tabel w jeden czytelny widok.

```
SELECT
  o.first_name + ' ' + o.last_name AS owner,
  a.name AS animal,
  v.visit_date,
  v.reason,
  t.name AS treatment,
  t.cost,
  p.medicine_name
FROM Owners o
JOIN Pets a ON o.owner_id = a.owner_id
JOIN Visits v ON a.animal_id = v.animal_id
JOIN Treatments t ON v.visit_id = t.visit_id
LEFT JOIN Prescriptions p ON t.treatment_id = p.treatment_id;
GO
```

	owner	animal	visit_date	reason	treatment	cost	medicine_name
1	Jan Kowalski	Rex	2025-04-10 10:00:00.000	szczepienie	szczepienie	50.00	NULL
2	Anna Nowak	Mruczek	2025-04-11 12:00:00.000	alergia	alergia	40.00	Dolvit Allergy
3	Jan Kowalski	Bella	2025-04-15 09:00:00.000	kontrola	kontrola	30.00	NULL
4	Piotr Zieliński	Kropka	2025-04-16 11:00:00.000	uraz łapy	uraz łapy	90.00	RemediCann
5	Piotr Zieliński	Kropka	2025-04-16 11:00:00.000	uraz łapy	uraz łapy	90.00	Meloksykam
6	Marzena Mazur	Kruszynka	2025-05-16 13:00:00.000	operacja	operacja	150.00	Antybiotyk VetCure
7	Marzena Mazur	Kruszynka	2025-05-16 13:00:00.000	operacja	operacja	150.00	Meloksykam
8	Tomasz Baran	Puszek	2025-05-16 11:00:00.000	diagnostyka	diagnostyka	15.00	Dolfos Vetcal D3
9	Robert Dąbrowski	Maks	2025-05-16 11:00:00.000	uraz łapy	uraz łapy	110.00	Beaphar Wound Maść
10	Katarzyna Niewiadoma	Kulka	2025-05-18 14:00:00.000	badanie ogólne	badanie ogólne	30.00	NULL

Rysunek 28. Przykład relacji - pełny widok pacjenta

- Procedura ExportAnimals_ToJson_BySpecies

Ta procedura służy do eksportowania danych zwierząt do formatu JSON. Dane są: grupowane według gatunku, konwertowane do formatu JSON, zapisywane w tabeli AnimalsJson. Procedura wykorzystuje: cursor, format JSON, transakcje, obsługę błędów.

Tabela AnimalsJson służy do przechowywania wygenerowanych danych JSON. Zawiera: identyfikator grupy, nazwę gatunku oraz dane zwierząt zapisane w formacie JSON.

```

CREATE TABLE AnimalsJson (
    group_id INT IDENTITY(1,1) PRIMARY KEY,
    species NVARCHAR(20),
    json_data NVARCHAR(MAX)
);
GO

```

```

CREATE PROCEDURE ExportAnimals_ToJson_BySpecies
AS
BEGIN

    DECLARE @species NVARCHAR(20);
    DECLARE @json NVARCHAR(MAX);

    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE species_cursor CURSOR FOR
        SELECT DISTINCT species
        FROM dbo.Pets;

        OPEN species_cursor;

        FETCH NEXT FROM species_cursor INTO @species;

        WHILE @@FETCH_STATUS = 0
        BEGIN

            SELECT @json =
            (
                SELECT
                    animal_id,
                    name,
                    breed,
                    birth_date,
                    weight,
                    gender
                FROM dbo.Pets
                WHERE species = @species
                FOR JSON PATH
            );

            INSERT INTO dbo.AnimalsJson (species, json_data)
            VALUES (@species, @json);

            FETCH NEXT FROM species_cursor INTO @species;
            END;

            CLOSE species_cursor;
            DEALLOCATE species_cursor;

            COMMIT TRANSACTION;

        END TRY

        BEGIN CATCH

            IF @@TRANCOUNT > 0
                ROLLBACK TRANSACTION;

            CLOSE species_cursor;
            DEALLOCATE species_cursor;

            PRINT 'Błąd podczas generowania JSON zwierząt';

        END CATCH

    END;
GO

```

- Sprawdzenie

```
EXEC ExportAnimals_ToJson_BySpecies;
SELECT * FROM AnimalsJson;
```

	group_id	species	json_data
1	1	chomik	[{"animal_id":5,"name":"Kruszynka","gender":"samica"}, {"animal_id":10,"name":"Kulka","weight":0.38,"gender":"samica"}, {"animal_id":11,"name":...
2	2	kot	[{"animal_id":2,"name":"Mruczek","breed":"dachowiec","weight":4.20,"gender":"samiec"}, {"animal_id":4,"name":"Kropka","breed":"perski","birth_d...
3	3	pies	[{"animal_id":1,"name":"Rex","breed":"labrador","birth_date":"2020-05-10","weight":30.50,"gender":"samiec"}, {"animal_id":3,"name":"Bella","bree...

Rysunek 29. Przykład tabeli AnimalsJson z formatem Json

Podsumowanie

Projekt przedstawia system bazy danych dla kliniki weterynaryjnej wykonany w SQL Server.

W projekcie zaimplementowano: relacyjny model bazy danych, klucze główne i obce, procedury składowane, funkcje, triggerzy, transakcje, obsługę błędów TRY/CATCH, kursory i eksport danych do formatu JSON.

System umożliwia:

- zarządzanie właścicielami i zwierzętami,
- obsługę wizyt weterynaryjnych,
- dodawanie zabiegów i recept,
- generowanie statystyk,
- kontrolę integralności danych,
- automatyczne logowanie operacji.

Projekt został przygotowany w sposób umożliwiający wielokrotne uruchamianie skryptu oraz zachowanie spójności danych.